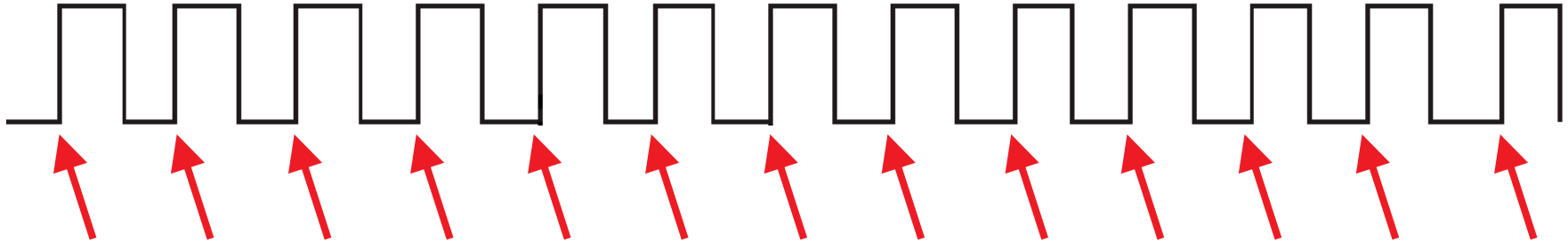
IoT 1

Les 3 – klokinstellingen, watchdog, debugger, interrupts, flankdetectie, …

Klokinstellingen

Basisklokinstellingen van de STM32F091RC

- **Klok**
  - Om een microcontroller ‘aan het werk te zetten’, moet je een **kloksignaal voorzien**.
  - Dat is een soort ‘blokgolf’ die bij iedere puls de microcontroller **een berekening laat maken**…
  - Het is dikwijls zo dat de instructie (berekening), start bij iedere ‘stijgende **flank**’ van dat kloksignaal.



Basisklokinstellingen van de STM32F091RC

- **Klok**
  - Onze microcontroller kan werken met een klok van **maximum 48MHz**.
  - Dat zijn 48 miljoen instructies/berekeningen per seconde…
  - Je kan werken met een extern ‘kristal’, of met interne RC-oscillatoren.
  - Belangrijk: **geen klok = geen berekeningen**!

Bron: https://www.flickr.com/photos/snazzyguy/4150705616/

Basisklokinstellingen van de STM32F091RC

- **Klok**
  - De instellingen i.v.m. de klok, gebeuren in de functie ***SystemClock_Config()***.

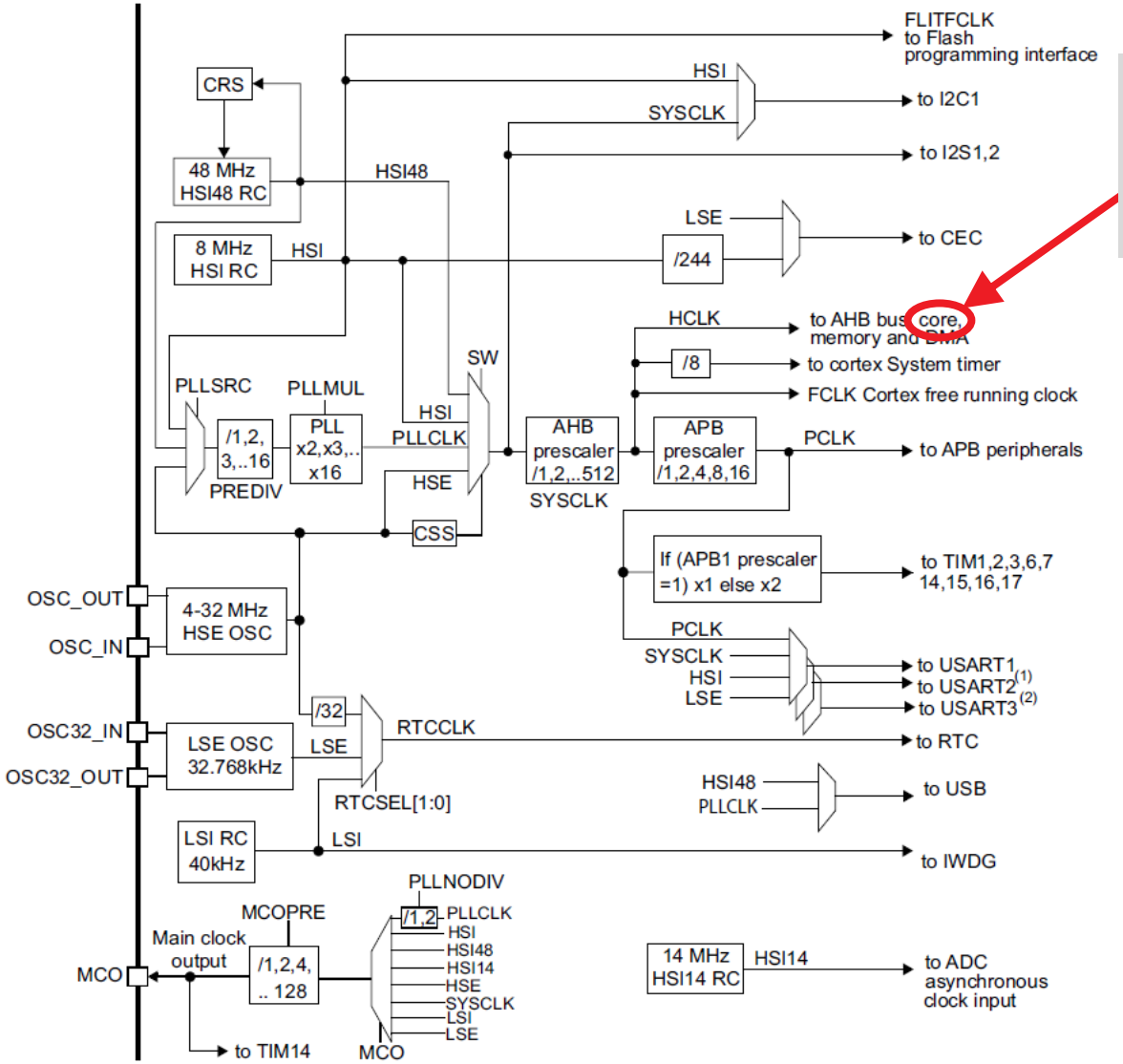
```
// Klokken instellen. Deze functie niet wijzigen, tenzij je goed weet wat je doet.
void SystemClock_Config(void)
{
  RCC->CR |= RCC_CR_HSITRIM_4;                          // HSITRIM op 16 zetten, dit is standaard (ook na reset).
  RCC->CR  |= RCC_CR_HSION;                             // Internal high speed oscillator enable (8MHz).
  while((RCC->CR & RCC_CR_HSIRDY) == 0);                // Wacht tot HSI zeker ingeschakeld is.

  RCC->CFGR &= ~RCC_CFGR_SW;                            // System clock op HSI zetten (SWS is status geupdatet door
  while((RCC->CFGR & RCC_CFGR_SWS) != RCC_CFGR_SWS_HSI); // Wachten to effectief HSI in actie is getreden.
```

Wat dit allemaal betekent, kan je volgen op de ‘clock tree’ **verderop** of in de reference manual.

Basisklokinstellingen van de STM32F091RC

- **Klok**

Figure 11. Clock tree (STM32F04x, STM32F07x and STM32F09x devices)



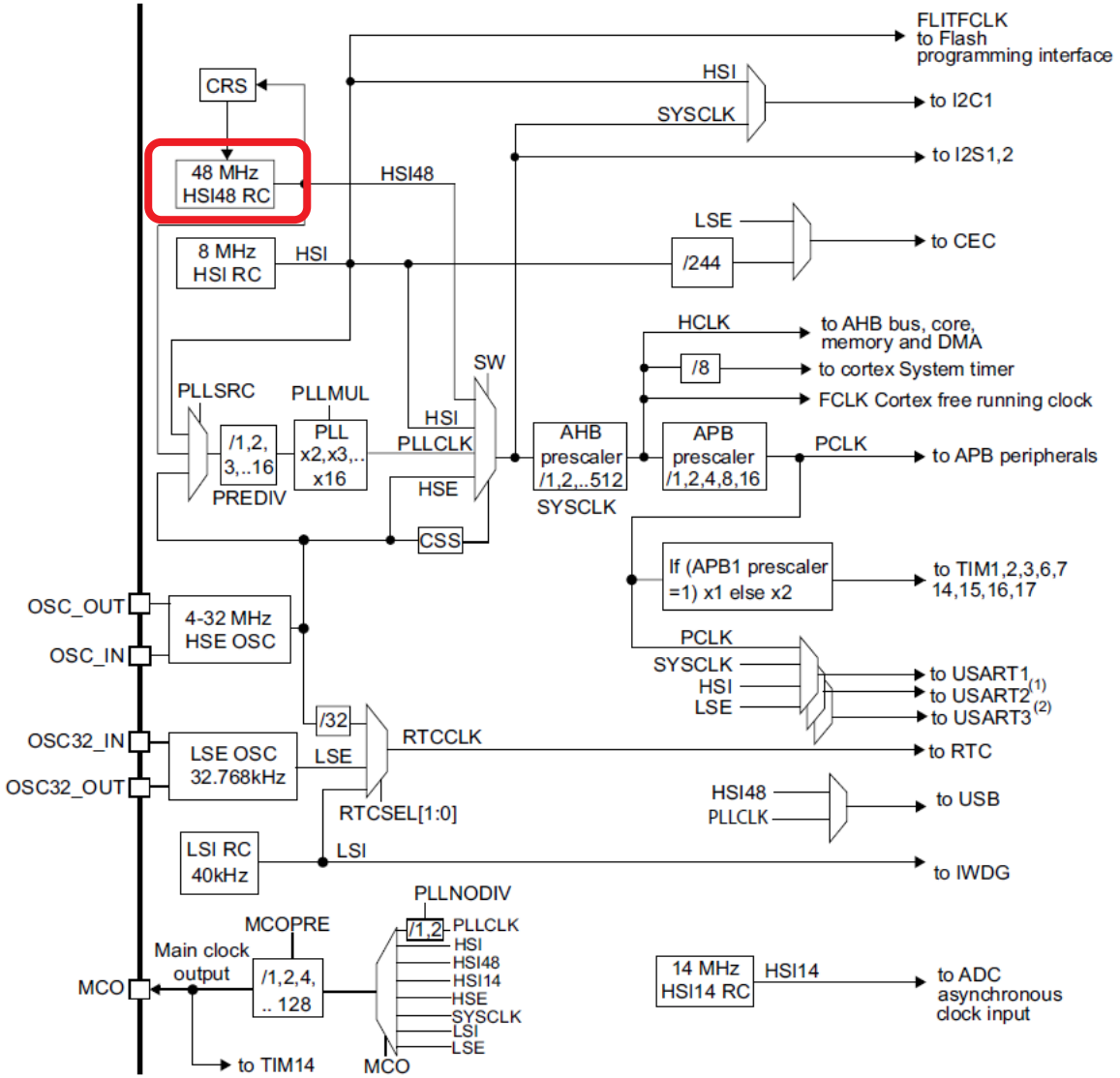
Hier worden de instructies uitgevoerd. Dit is dus **dé plaats** om onze klok van 48MHz naartoe te sturen!

Bron: STM32F0 Reference Manual – RM0091 (st.com)

Basisklokinstellingen van de STM32F091RC

- **Klok**

Figure 11. Clock tree (STM32F04x, STM32F07x and STM32F09x devices)



Er is een 48MHz interne RC oscillator beschikbaar, maar die is **niet erg stabiel**.

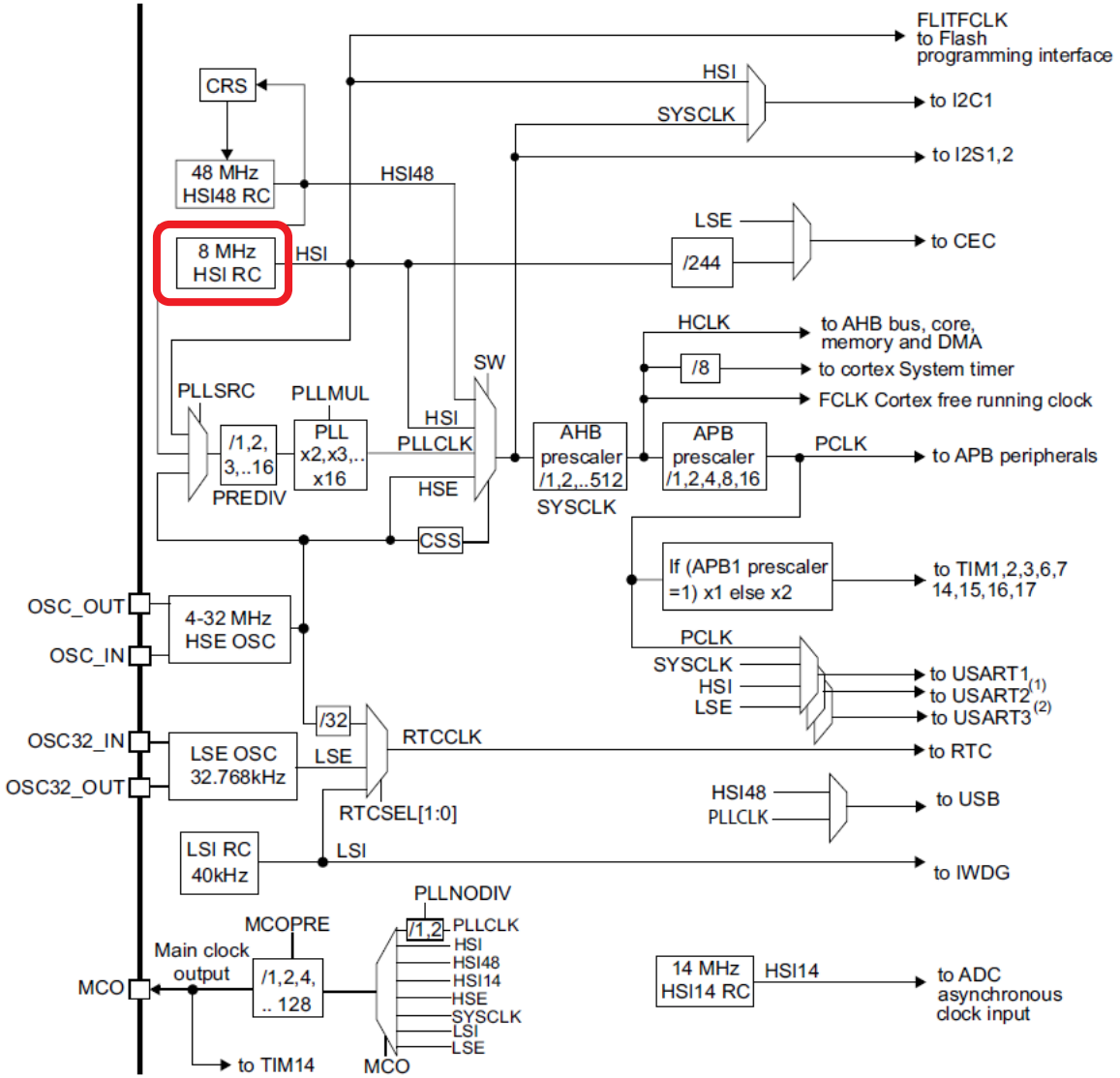
We wensen die **niet** te gebruiken.

Bron: STM32F0 Reference Manual – RM0091 (st.com)

Basisklokinstellingen van de STM32F091RC

- **Klok**

Figure 11. Clock tree (STM32F04x, STM32F07x and STM32F09x devices)



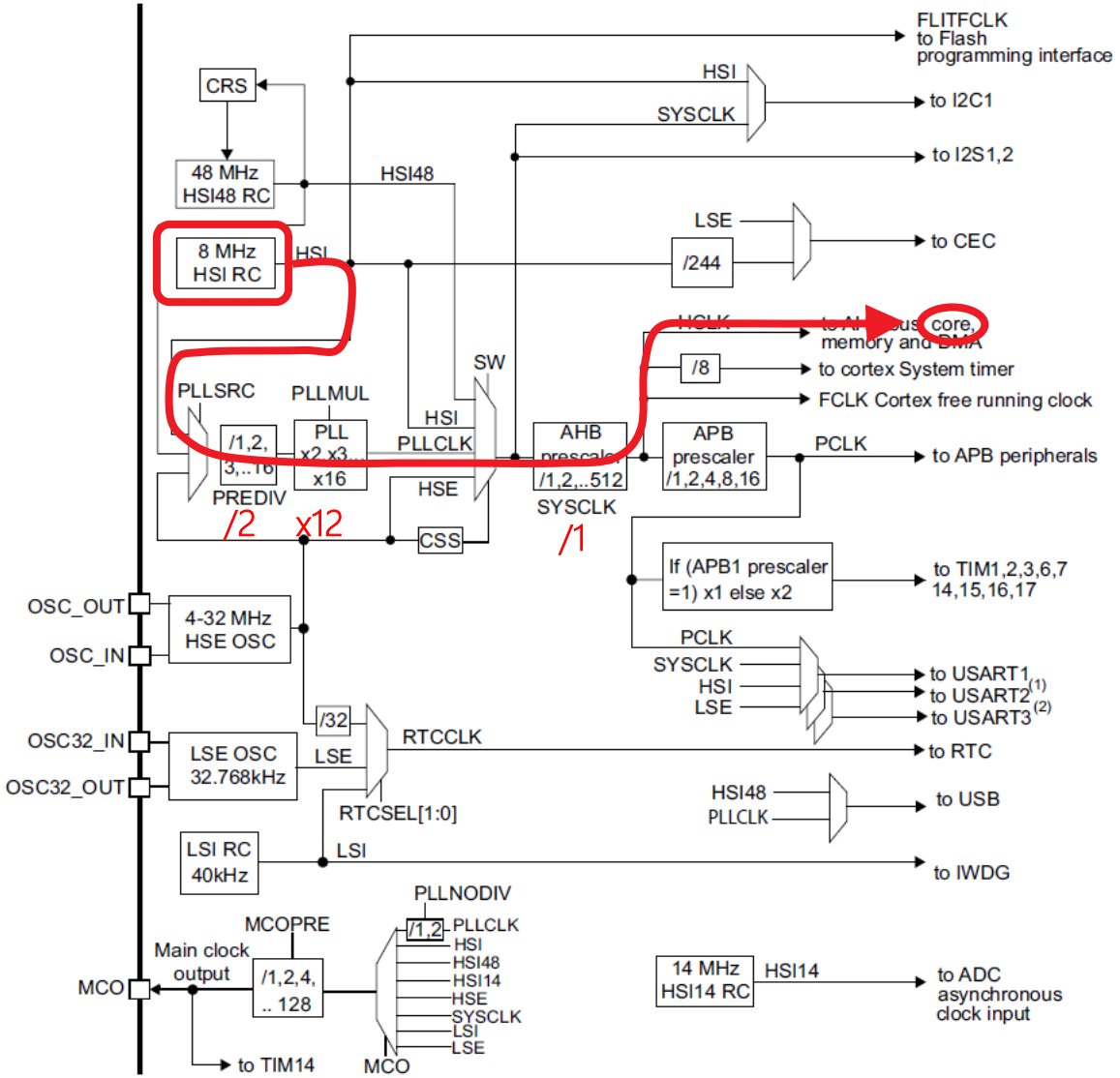
Start daarom vanaf de 8MHz klok (interne RC oscillator), die je kan '**vermenigvuldigen**' tot een klok van 48MHz bij de 'core'.

Bron: STM32F0 Reference Manual – RM0091 (st.com)

Basisklokinstellingen van de STM32F091RC

- **Klok**

Figure 11. Clock tree (STM32F04x, STM32F07x and STM32F09x devices)



Kan je **het pad maken**/volgen tot aan de CPU core, met 48MHz?

Let op de delers en vermenigvuldigers (PLL)!

Zie je ondertussen de **link met** de `SystemClock_Config()` functie?

Bron: STM32F0 Reference Manual – RM0091 (st.com)

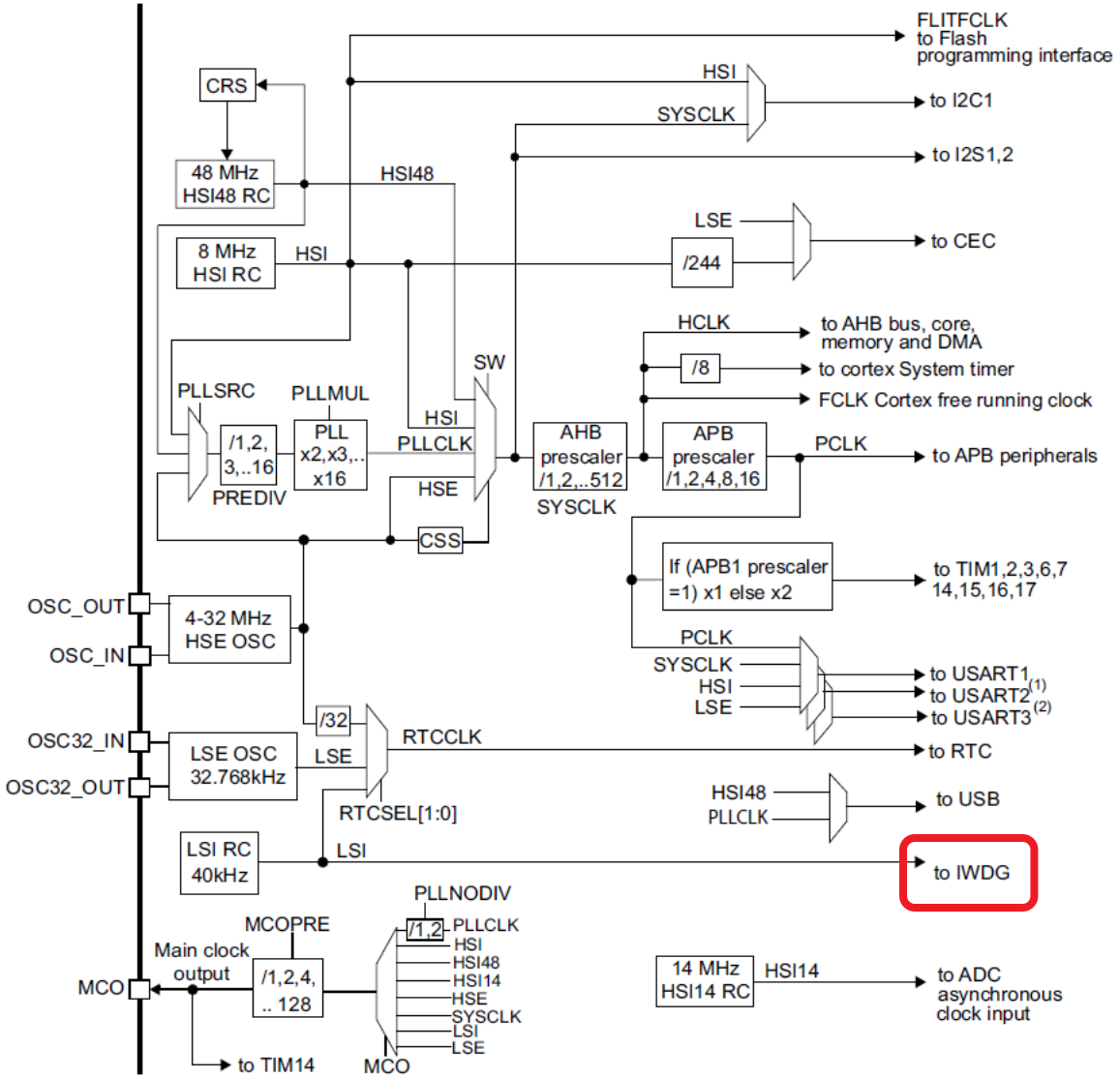
Watchdog timer

Watchdog timer

- Dikwijls hebben de microcontrollertoepassingen maar een heel **beperkte (of geen) user interface**.
- Dat kan het lastig maken om de **goede werking** van je programma te **monitoren** of bij te sturen.
- Daarom wordt soms gebruik gemaakt van een watchdog timer.
- **Een watchdog timer is een soort 'waakhond'.**
- Wanneer de timer van die **watchdog** niet **tijdig gereset** wordt, zal die watchdog **de gehele microcontroller resetten**!
- Dat in de **hoop** dat het programma dan opnieuw goed loopt…

Watchdog timer

Figure 11. Clock tree (STM32F04x, STM32F07x and STM32F09x devices)



Ook onze microcontroller heeft een watchdog timer.

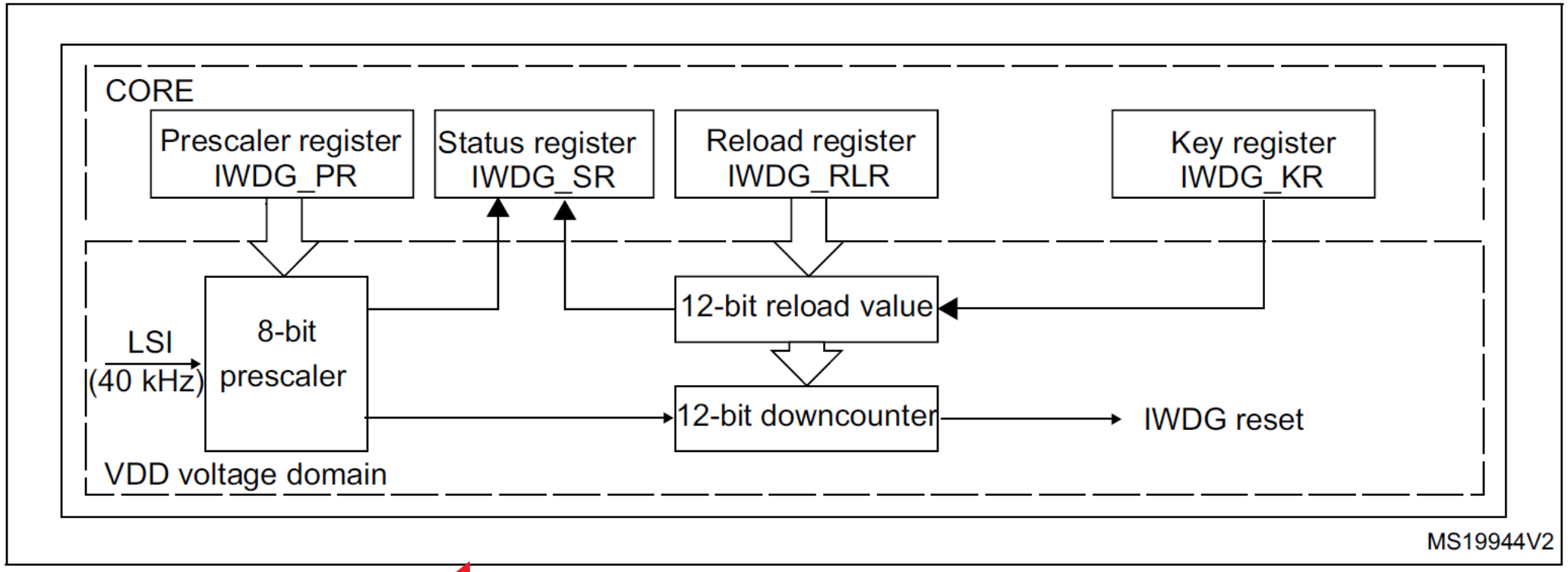
Die heet 'Independant Watchdog' (**IWDG**).

Met welke klok wordt die gevoed? **Waarom**?

Bron: STM32F0 Reference Manual – RM0091 (st.com)

Watchdog timer

Figure 208. Independent watchdog block diagram



Wat is er allemaal te zien?

Bron: STM32F0 Reference Manual – RM0091 (st.com)

Watchdog timer

- Het **instellen** en gebruiken van de IWDG, vereist het volgen van een specifieke **procedure**. Deze vind je in de reference manual…

Configuring the IWDG when the window option is disabled

When the window option it is not used, the IWDG can be configured as follows:

1. Enable the IWDG by writing 0x0000 CCCC in the IWDG_KR register.
2. Enable register access by writing 0x0000 5555 in the IWDG_KR register.
3. Write the IWDG prescaler by programming IWDG_PR from 0 to 7.
4. Write the reload register (IWDG_RLR).
5. Wait for the registers to be updated (IWDG_SR = 0x0000 0000).
6. Refresh the counter value with IWDG_RLR (IWDG_KR = 0x0000 AAAA)

For code example refer to the Appendix section *A.15.1: IWDG configuration code example*.

Bron: STM32F0 Reference Manual – RM0091 (st.com)

Watchdog timer

- In **C-code** wordt dat dan…

Waarom deelt de klok 256 keer? Zoek het uit via µVision, gecombineerd met de reference manual!

```c
void InitWatchdog(void)
{
  IWDG->KR = 0x0000CCCC;     // Enable de watchdog.
  IWDG->KR = 0x00005555;     // Schrijftoegang tot registers van watchdog verschaffen.
  IWDG->PR = IWDG_PR_PR;     // /256 => bron klok is 40kHz => 6,4ms per stap.
  IWDG->RLR = 1000;          // 1000 x 6,4ms = 6,4s...  (telt af naar 0)
  while(IWDG->SR != 0);      // Wacht tot bovenstaande registers geüpdatet zijn.
  IWDG->KR = 0x0000AAAA;     // Reset de watchdog en start hem.
}

void ResetWatchdog(void)
{
  IWDG->KR = 0x0000AAAA;     // Reset de watchdog.
}
```

Watchdog timer

- **Demo**
  - Maak gebruik van ‘watchdog.h’ en ‘watchdog.c’.
  - Bij inschakelen laat je de **LED’s knipperen** (als bewijs van reset).
  - Reset de IWDG timer wanneer je op **SW1** of de **User Button** drukt.

  - Zie je de microcontroller resetten wanneer je **te lang wacht** met het drukken op de knop? ;-)

De µVision debugger gebruiken

De µVision debugger gebruiken

- **Wat**?
  - Programmeren = **fouten maken**…
  - Ga efficiënt te werk om die fouten **op te sporen**!
  - Doe dat door ‘break points’ te zetten en de **debugger** te starten.
  - **Break point** = een plaats waar je jouw programma wil pauzeren om alle variabelen en registers te bekijken (op ‘mensen-tempo’).
  - De on board **ST-link** programmer is ook een debugger!

De µVision debugger gebruiken

- **Hoe**?
  - Open een **µVision** project en **klik** in de linker marge om een break point toe te voegen.
  - **Start** de debugger.
  - Open een '**Watch Window**' om **variabelen** te bekijken.
  - Of gebruik de '**System Viewer**' om **registers** te bekijken.

OEFEN HET DEBUGGEN ZELF UIT!

DIT NIET KUNNEN, IS GEEN OPTIE!

hogeschool vives

De µVision debugger gebruiken

C:\temp\Nucleo-template 1\template.uvprojx - µVision

File Edit View Project Flash Debug Peripherals Tools SVCS Window Help

main.c | leds.c | stm32f091xc.h | buttons.c | ad.c | startup_stm32f091xc.s

```
39  // Laten weten dat we opgestart zijn, via de USART2 (USB).
40  StringToUsart2("Reboot\r\n");
41
42  // Oneindige lus starten.
43  while (1)
44  {
45    adValue = (uint8_t)(GetAdValue() >> 4);
46
47    ByteToLeds(adValue);
48    // of
49    //ByteToLevel(adValue);
50
51    WaitForMs(100);
52  }
53
```

Watch 1

Name	Value	Type
adValue	0x77 'w'	uchar
<Enter expression>		

De µVision debugger gebruiken

C:\temp\Nucleo-template 1\template.uvprojx - µVision

File Edit View Project Flash Debug Peripherals Tools SVCS Window Help

main.c | leds.c | stm32f091xc.h | buttons.c | ad.c | startup_stm32f091xc.s

```c
#include "stm32f091xc.h"
#include "ad.h"

// AD-module initialiseren
void InitAd(void)
{
  // Poort A 0 als analoge ingang (potentiometer op het Nucleo Extension Shield).
  GPIOA->MODER |= GPIO_MODER_MODER0;

  // AD-module van klok voorzien.
  RCC->APB2ENR |= RCC_APB2ENR_ADC1EN;

  // Right alignment (optioneel).
  ADC1->CFGR1 &= ~ADC_CFGR1_ALIGN;

  // AD enable'n.
  ADC1->CR |= ADC_CR_ADEN;
  while ((ADC1->ISR & ADC_ISR_ADRDY) == 0);

  // AD-kanaal 0 kiezen (verbonden met de connector voor analog input).
  ADC1->CHSELR = ADC_CHSELR_CHSEL0;

  // Tragere sample rate kiezen (voor hoogimpedante analoge bronnen).
  ADC1->SMPR |= ADC_SMPR_SMP_0 | ADC_SMPR_SMP_1 | ADC_SMPR_SMP_2;
}
```

GPIOA

Property	Value
MODER	0x280104A3
MODER15	0x00
MODER14	0x02
MODER13	0x02
MODER12	0x00
MODER11	0x00
MODER10	0x00
MODER9	0x00
MODER8	0x01
MODER7	0x00
MODER6	0x00
MODER5	0x01
MODER4	0x00
MODER3	0x02
MODER2	0x02
MODER1	0x00
MODER0	0x03

hogeschool vives

De µVision debugger gebruiken

Niet duidelijk? Beetje snel gegaan?

Bekijk zeker ook de **instructievideo** op Toledo i.v.m. debuggen in µVision! Voor je het weet, ben je expert in debuggen ;-).

Interrupts

Eenvoudige interrupts

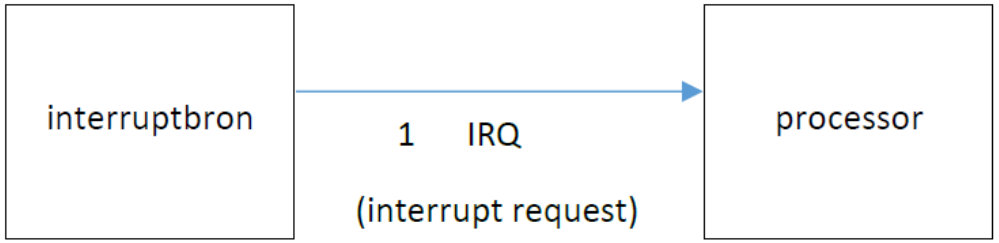
- **Wat** is een interrupt?

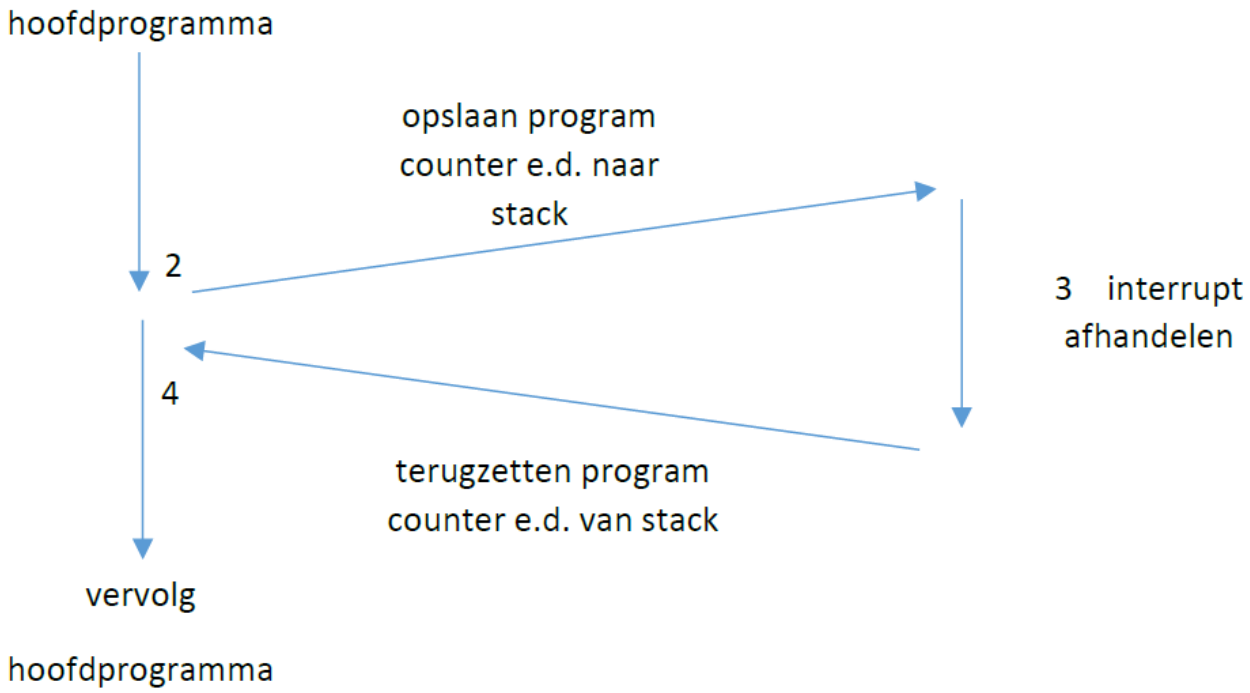
Een interrupt is een **verzoek aan de processor om onmiddellijk het hoofdprogramma te verlaten** en een ‘interrupt routine’ af te handelen.

Die ‘interrupt handler’ verwerkt de situatie, waarna de processor **opnieuw** met het hoofdprogramma **kan verder gaan**.

To interrupt = onderbreken…

Eenvoudige interrupts





Eenvoudige interrupts

- **Waarom** een interrupt?
    - Om **heel snel in te spelen** op een verzoek van de ‘interrupt aanvrager’.
    - Op die manier kan je bijvoorbeeld op een hoge snelheid bytes ontvangen via de seriële poort, waarvan je **op voorhand niet weet ‘wanneer’** die zullen aankomen…

Eenvoudige interrupts

- **Hoe?**
  - Via de **Nested Vectored Interrupt Controller** (NVIC).
  - Dit is de **schakel tussen** de interruptbron en de CPU.
  - Kan **32 interruptkanalen** in goede banen leiden.
  - Daarbij zijn **4 programmeerbare prioriteitniveaus** (0 tot 3). 0 is **meest** belangrijk, 3 is **minst** belangrijk.
  - Iedere interruptbron **heeft ook een niet-programmeerbaar prioriteitniveau**.
  - **Twee** interrupts **op hetzelfde moment**, met dezelfde programmeerbare prioriteit => afhandeling volgens niet-programmeerbare prioriteit.

Eenvoudige interrupts

- Priority level **voorbeeld**
  - Op **hetzelfde moment** een interrupt van EXTI1 en SPI1, waarvan de programmable interrupt priority beide op 0 stonden?
  - **Eerst** wordt de **EXTI1** interrupt afgehandeld, **daarna** de **SPI1** interrupt.

  - **Waarom**? EXTI1 heeft niet-programmeerbaar prioriteitniveau van 12. Bij SPI1 is dat 32 (zie volgende slide).

Eenvoudige interrupts

- Priority level **voorbeeld**

Table 36. Vector table (continued)

Position	Priority	Type of priority	Acronym	Description	Address
3	10	settable	FLASH	Flash global interrupt	0x0000 004C
4	11	settable	RCC_CRS	RCC and CRS global interrupts	0x0000 0050
5	12	settable	EXTI0_1	EXTI Line[1:0] interrupts	0x0000 0054
6	13	settable	EXTI2_3	EXTI Line[3:2] interrupts	0x0000 0058
7	14	settable	EXTI4_15	EXTI Line[15:4] interrupts	0x0000 005C
8	15	settable	TSC	Touch sensing interrupt	0x0000 0060
9	16	settable	DMA_CH1	DMA channel 1 interrupt	0x0000 0064
10	17	settable	DMA_CH2_3 DMA2_CH1_2	DMA channel 2 and 3 interrupts DMA2 channel 1 and 2 interrupts	0x0000 0068
11	18	settable	DMA_CH4_5_6_7 DMA2_CH3_4_5	DMA channel 4, 5, 6 and 7 interrupts DMA2 channel 3, 4 and 5 interrupts	0x0000 006C
12	19	settable	ADC_COMP	ADC and COMP interrupts (ADC interrupt combined with EXTI lines 21 and 22)	0x0000 0070
13	20	settable	TIM1_BRK_UP_TRG_COM	TIM1 break, update, trigger and commutation interrupt	0x0000 0074
14	21	settable	TIM1_CC	TIM1 capture compare interrupt	0x0000 0078
15	22	settable	TIM2	TIM2 global interrupt	0x0000 007C
16	23	settable	TIM3	TIM3 global interrupt	0x0000 0080
17	24	settable	TIM6_DAC	TIM6 global interrupt and DAC underrun interrupt	0x0000 0084
18	25	settable	TIM7	TIM7 global interrupt	0x0000 0088
19	26	settable	TIM14	TIM14 global interrupt	0x0000 008C
20	27	settable	TIM15	TIM15 global interrupt	0x0000 0090
21	28	settable	TIM16	TIM16 global interrupt	0x0000 0094
22	29	settable	TIM17	TIM17 global interrupt	0x0000 0098
23	30	settable	I2C1	I^2C1 global interrupt (combined with EXTI line 23)	0x0000 009C
24	31	settable	I2C2	I^2C2 global interrupt	0x0000 00A0
25	32	settable	SPI1	SPI1 global interrupt	0x0000 00A4

Bron: STM32F0 Reference Manual – RM0091 (st.com)

Eenvoudige interrupts

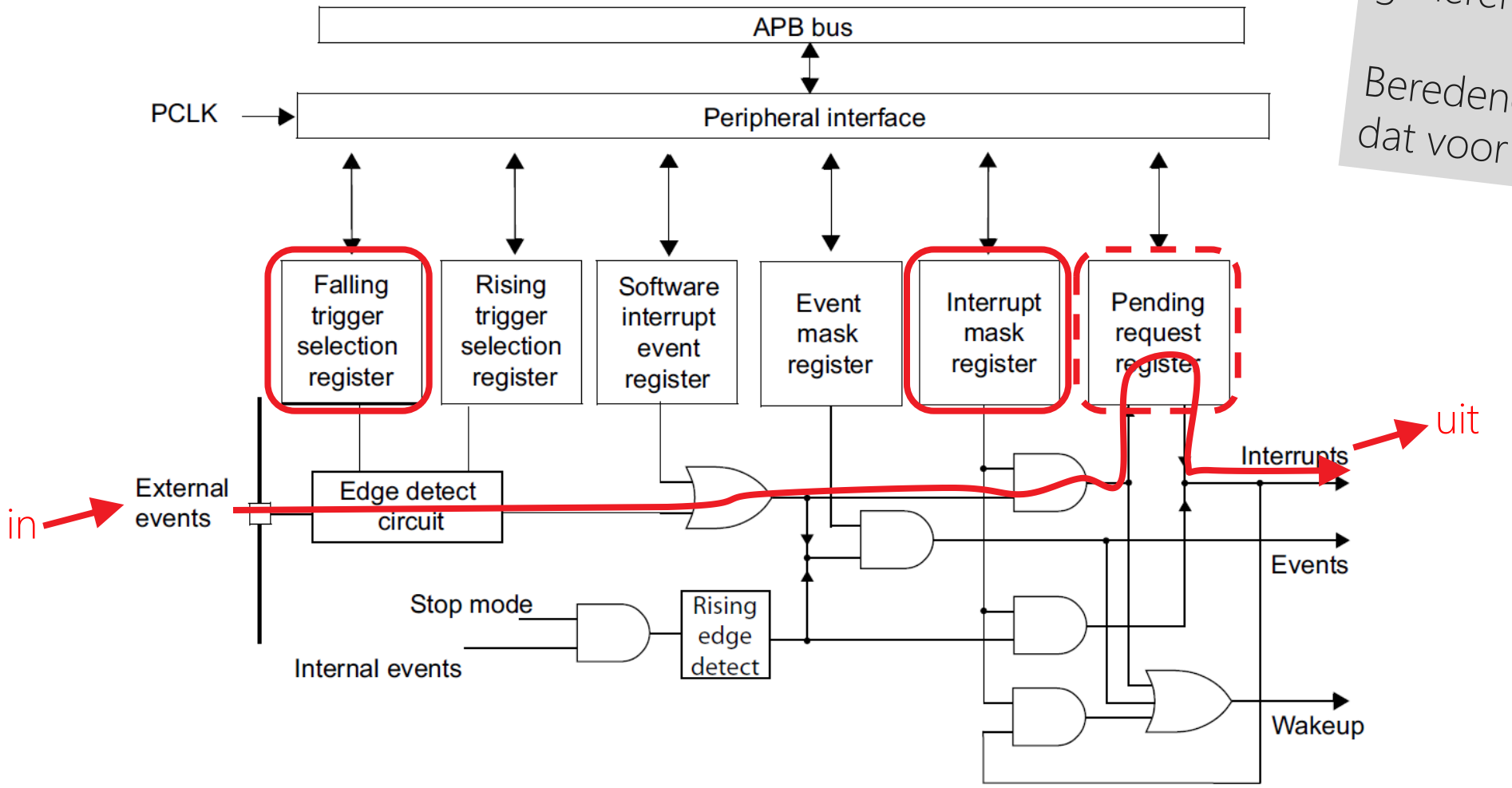
- Priority level **voorbeeld**
  - Wil je **toch eerst SPI1** afhandelen? Zet dan de programmable interrupt priority voor de EXTI1 op 1.

```
NVIC_SetPriority(EXTI0_1_IRQn, 0);
NVIC_SetPriority(SPI1_IRQn, 0);
```

```
NVIC_SetPriority(EXTI0_1_IRQn, 1);
NVIC_SetPriority(SPI1_IRQn, 0);
```

Eenvoudige interrupts

- Basisinstelling **External** Interrupt



Welk **pad** kan een externe trigger volgen om een interrupt te genereren?

Beredeneer dat voor SW2.

Bron: STM32F0 Reference Manual – RM0091 (st.com)

Eenvoudige interrupts

- Basisinstelling **External** Interrupt

Neem het schema van het NES erbij, om te kijken aan welke pin SW2 gekoppeld is.

```
34  // Interrupt instellen.
35  // SYSCFG clock enable.
36  RCC->APB2ENR |= RCC_APB2ENR_SYSCFGEN;
37
38  // PA4 (SW2) koppelen aan EXTI 4.
39  SYSCFG->EXTICR[1] |= SYSCFG_EXTICR2_EXTI4_PA;
40
41  // Falling edge detecteren.
42  EXTI->FTSR = EXTI->FTSR | EXTI_FTSR_TR4;
43
44  // Interrupt toelaten.
45  EXTI->IMR = EXTI->IMR | EXTI_IMR_MR4;
46
47  // Eén van de 4 prioriteiten kiezen. Als er twee
48  // interrupts zijn met dezelfde prioriteit, wordt
49  // eerst diegene afgehandeld die de laagste
50  // 'position' heeft (zie vector table NVIC in datasheet).
51  NVIC_SetPriority(EXTI4_15_IRQn,0);
52
53  // Interrupt effectief toelaten.
54  NVIC_EnableIRQ(EXTI4_15_IRQn);
55
56  // Oneindige lus starten.
57  while (1)
58  {
59    // Doe hier eens niets....!
60  }
```

Eenvoudige interrupts

- Basisinstelling **External** Interrupt

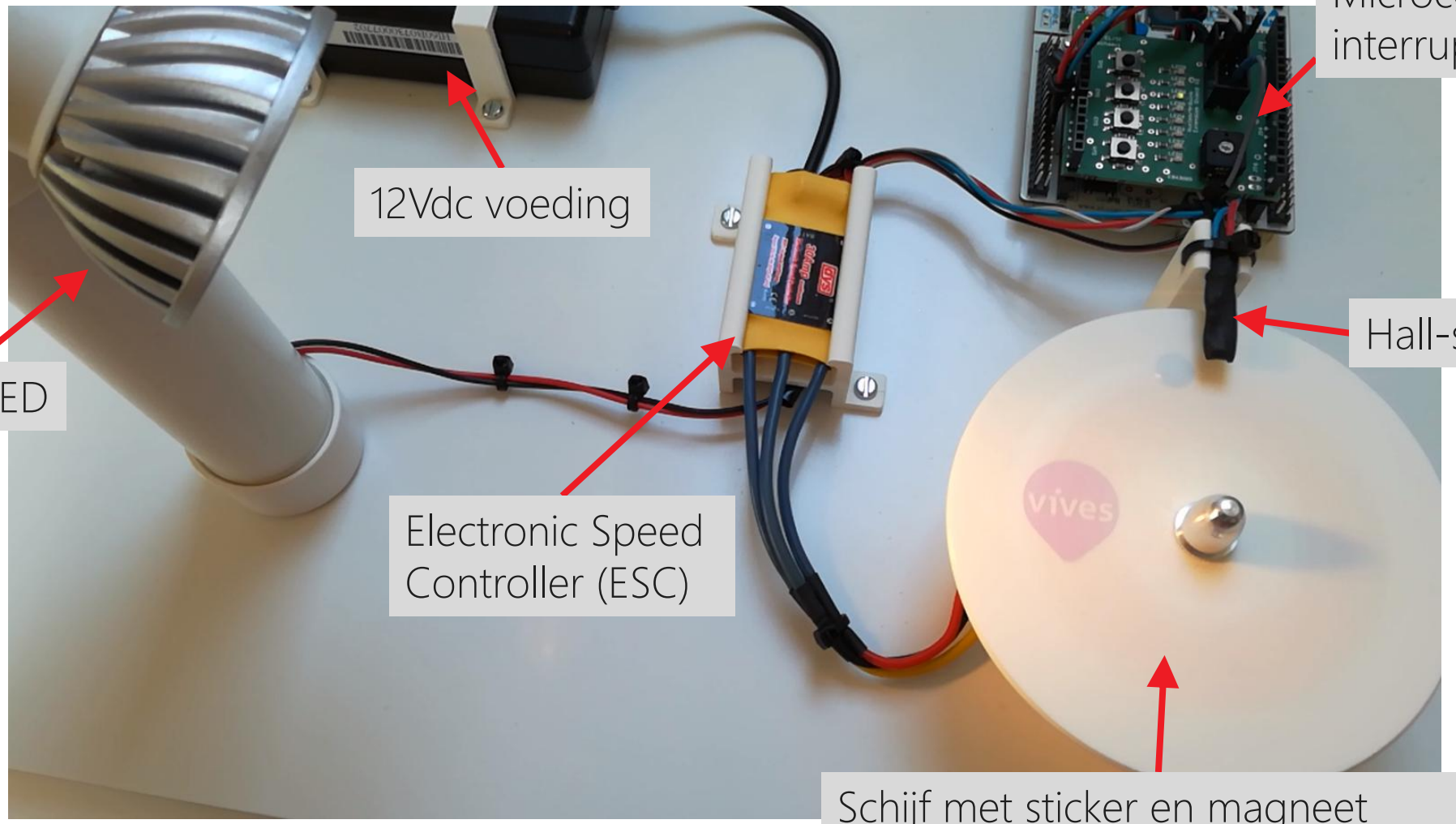
Zoek uit in de reference manual hoe je de 'pending vlag' kan wissen!

```
63 // Interrupt handler aanmaken. De definities van
64 // de handler kan je vinden in startup_stp32f091xc.s.
65 void EXTI4_15_IRQHandler(void)
66 {
67   // Als het een interrupt is van PA4, de LED's inschakelen.
68   if((EXTI->PR & EXTI_PR_PR4) == EXTI_PR_PR4)
69   {
70     // Interrupt (pending) vlag wissen door
71     // een 1 te schrijven...
72     EXTI->PR |= EXTI_PR_PR4;
73
74     ByteToLeds(255);
75   }
76 }
```

Kan je nog steeds lijn 68 <u>volledig</u> ontleden en begrijpen?

Demo: stroboscoopopstelling

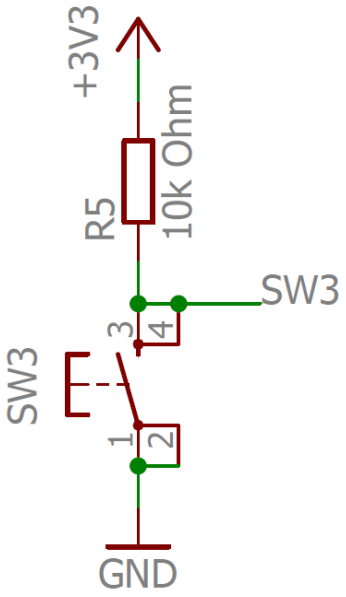
Demo: stroboscoopopstelling



Demo: flankdetectie

Demo: flankdetectie

- Soms wil je het **indrukken** van een knop detecteren op het moment van de **wijziging van toestand**. Zo'n wijziging is een 'flank' en kan stijgend of dalend zijn.
- **Welke flanken** genereren we als we op één van de vier drukknoppen drukken van het **NES**?
- Neem er het **schema** bij en verklaar.
- Houd rekening met '**dender**'…



Demo: flankdetectie

- Door het **trillen van de drukknoppen**, kan een ‘dendereffect’ ontstaan. Hierbij opent en sluit de schakelaar meer dan één keer op hele korte termijn… Niet leuk, maar onvermijdelijk!

Demo: flankdetectie

```
34  // Oneindige lus starten.
35  while (1)
36  {
37    // Wordt SW1 ingedrukt?
38    if(SW1Active())
39    {
40      // SW1 wordt ingedrukt.
41
42      // Is het de eerste keer dat je hier passeert nadat SW1 ingedrukt werd?
43      if(sw1Help == true)
44      {
45        // Knop is net ingedrukt.
46        // Er komt 0V binnen op de microcontroller.
47        // Sla die info op voor gebruik in de toekomst.
48        sw1Help = false;
49
50        // Doe hier wat je wil doen bij een dalende flank van SW1.
51        ToggleLed(1);
52      }
53    }
54    else
55      // SW1 wordt niet ingedrukt.
56      // Er komt 3.3V binnen op de microcontroller (via de pull-up weerstand).
57      // Sla die info op voor gebruik in de toekomst.
58      sw1Help = true;
59
60    // Even wachten zodat je geen last hebt van dender.
61    WaitForMs(10);
62  }
```

Your turn now!

- Is er nog **wat resterende** tijd?
  - Maak **Theorieopdracht 2 – bit bewerkingen**.
  - Maak **Theorieopdracht 3 – pin als output** plaatsen.

  - Ga **zelf aan de slag** om **interrupts** te verwerken, of maak een **flankdetectie** voor de vier knoppen…